School of Informatics & Cybersecurity - Assessment Submission Coversheet

Student Information

Student Number:	B00162757
Student Name:	Cathal Farrell

Module Information

Module Title:	Web Framework Development
Module Code:	COMP H3037

Assessment Information

Assessment Title:	Project

Declaration

☑ I confirm that I have read and understood the University's Academic Integrity Policy.

https://www.tudublin.ie/explore/about-the-university/academic-affairs/academic-quality-assurance-and-enhancement/academic-integrity/

☑ I have completed the TU Dublin AI Driving License theory test.

https://www.tudublin.ie/explore/about-the-university/academic-affairs/lta/lta-resources/artificial-intelligence-driving-license/

☑ I declare that this assessment submission is my own work.

☑ I confirm that I have read and understood the AI traffic light system used as part of this assessment brief and I have adhered to the level of AI usage allowed for this assessment.

Student Signature: Cathal Farrell

Date: 27/04/2026

Web Framework Development Project:

Retail – Training

Submitted by: Cathal Farrell, B00162757

Table of Contents

Contents

School of Informatics & Cybersecurity - Assessment Submission Coversheet 1
Student Information 1
Module Information 1
Assessment Information 1
Declaration 1
Table of Contents 3
1. Business Understanding 4
Business Requirements 4
2. Use Cases 5
3. Database Design 7
4. Django Framework Implementation 8
Model: 8
Implementation 8
Relations 8
View 10
Template 12
5. Project Testing 14
6. Media Sources 14
7. Notes for use of web server 14

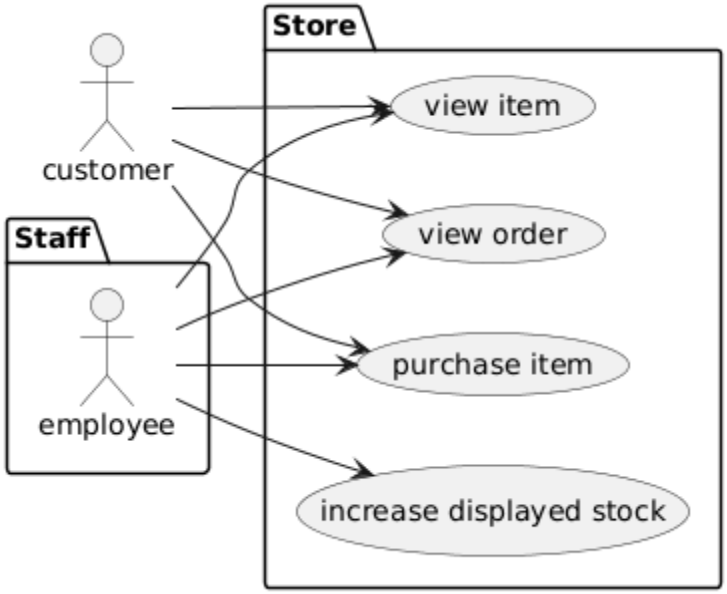
1. Business Understanding

An e-commerce site that allows consumers to order items to be collected at a local shop and view orders that have not yet been collected.
The website will also provide training for employees as they are preparing to take on new roles within the company.

Business Requirements

- User's can order new items to be delivered to their store of choice.
- User's can check the details of orders that have not been collected.
- Employees' roles can be managed quickly and easily.
- Employees receive a tutorial and are expected to pass a test upon receiving a new role. Unpassed tests are removed from employee's notifications upon losing a role.

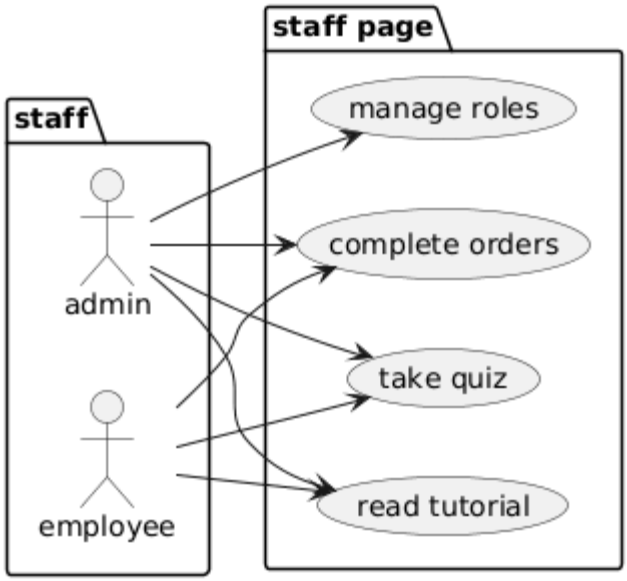
2. Use Cases



Any user can…

- View selection of items for sale.
- Purchase a product that is for sale.
- View an order they have previously made.

Staff can also increase the databases stock count for each product.

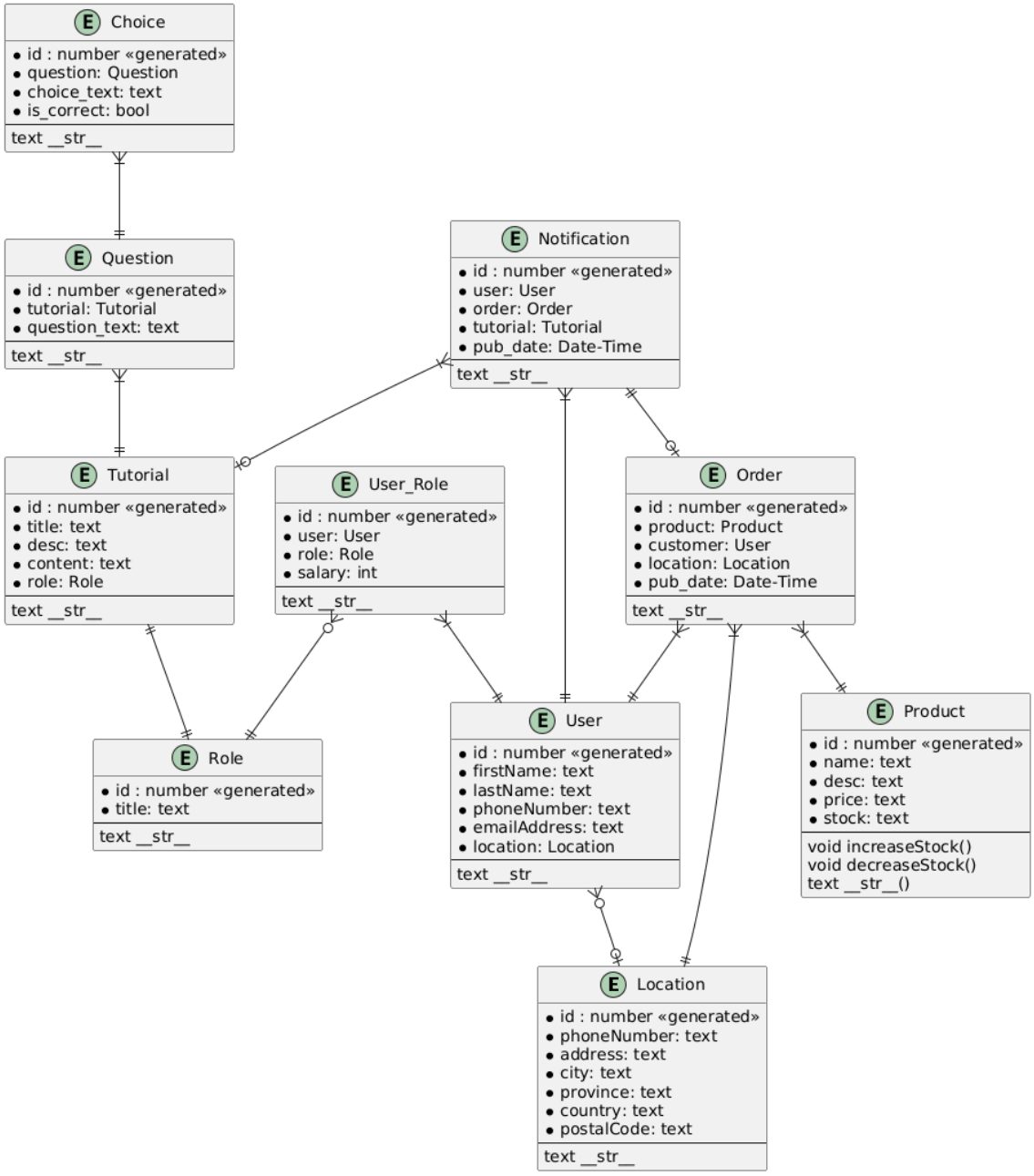


All staff users can…

- Complete outstanding orders.
- Read tutorials after receiving a new job position
- Take quizzes on said tutorials.

Admin users can also manage the roles of all users.

3. Database Design



4. Django Framework Implementation

Model:

```python
from django.db import models

# Create your models here.

class Tutorial(models.Model):
    title = models.CharField(max_length=200)
    description = models.CharField(max_length=600)
    content = models.CharField(max_length=20000)
    roleID = models.ForeignKey('store.Role', on_delete=models.CASCA

    def __str__(self):
        return self.title

class Question(models.Model):
    tutorial = models.ForeignKey(Tutorial, on_delete=models.CASCADE
    question_text = models.CharField(max_length=200)

    def __str__(self):
        return self.question_text

class Choice(models.Model):
    question = models.ForeignKey(Question, on_delete=models.CASCADE
    choice_text = models.CharField(max_length=200)
    is_correct = models.BooleanField()

    def __str__(self):
        return self.choice_text
```

Figure 1: code from staff:model.py

Implementation

Each model was used as described in the ERD with minor changes to names of attributes, such as adding ID to the end of foreign keys' names.

Relations

Tutorial -> Role:
1 to 1, each job position has a dedicated tutorial and quiz.

Tutorial -> Question:
1 to many, each tutorial has a set of questions that are asked in a quiz after reading.

Question -> Choice:
to many, each question has a few choices to select from when answering.

Location -> User:
1 to many, many employees can share a store location in which they are employed.

User_Role -> User:
1 to many, each role can be occupied by multiple people.

User_Role -> Role:
1 to many, each employee can hold multiple job positions.

Order -> Location:
1 to many, many orders can be delivered to the same store.

Order -> User:
1 to many, many orders can be ordered by the same customer.

Order -> Product:
1 to many, many order can be made for the same product.

View

```python
from .models import Product, Location, User, Order, Notification


def index(request):
    products_list = Product.objects.order_by()
    user = User.objects.get(firstName="user", lastName="1")
    notifications_list = Notification.objects.filter(userID=user)
    template = loader.get_template("store/index.html")
    context = {"products_list": products_list, "notifications_list"
    return HttpResponse(template.render(context, request))

def detail(request, product_id):
    product = Product.objects.get(pk=product_id)
    template = loader.get_template("store/product.html")
    context = {"product": product}
    return HttpResponse(template.render(context, request))

def purchase(request, product_id):
    product = Product.objects.get(pk=product_id)
    locations_list = Location.objects.order_by()
    template = loader.get_template("store/purchase.html")
    context = {"product": product, "locations_list": locations_list
    return HttpResponse(template.render(context, request))
```

Figure 2: code snippet of display pages from store:views.py

```python
def sale(request, product_id):
    product = get_object_or_404(Product, pk=product_id)
    try:
        username = User.objects.get(firstName=request.POST["fname"],
        location = Location.objects.get(pk=request.POST["locations"])
    except (KeyError, User.DoesNotExist):
        # Redisplay the question voting form.
        return render(
            request,
            "store/product.html",
            {
                "product": product,
                "error_message": "An account with that username does
            },
        )
    else:
        product.decreaseStock(qty=1)
        product.save()

        order = Order(productID=product, customerID=username, date=ti
        order.save()

        notification = Notification(orderID = order, userID=usernam
        notification.save()

        # Always return an HttpResponseRedirect after successfully de
        # with POST data. This prevents data from being posted twice
        # user hits the Back button.
        return HttpResponseRedirect(reverse("store:index"))
```

Figure 3: code snippet of function page from store:views.py

```python
app_name = 'store'

urlpatterns = [
    path("", views.index, name="index"),
    path("<int:product_id>/", views.detail, name="detail"),
    path("<int:product_id>/purchase/", views.purchase, name="purcha
    path("<int:product_id>/sale/", views.sale, name="sale"),
    path("order/<int:order_id>/", views.order, name="order"),
]
```

Figure 4: code snippet from store:urls.py

Template

```
<a href="/store/"><h1>Home</h1></a>

{% if products_list %}
    <h3>Products:</h3>
    <ul>
    {% for product in products_list %}
        {% if product.stock > 0 %}
        <li><a href="/store/{{ product.id }}/">€{{product.price}} -
        {% endif %}
    {% endfor %}
    </ul>
{% else %}
    <p>No products are available.</p>
{% endif %}

{% if notifications_list %}
    <h3>Notifications:</h3>
    <ul>
    {% for notification in notifications_list %}
        {% if notification.orderID %}
            <li><a href="/store/order/{{ notification.orderID.id }}
        {% else %}
            <li><a href="/staff/tutorial/{{ notification.tutorialID
        {% endif %}
    {% endfor %}
    </ul>
{% else %}
    <p>No Notifications are available.</p>
{% endif %}
```

Figure 5: code snippet of store's index page from store:index.py

Home

Products:

- €1199 - iPhone 15 Pro Max
- €899 - Samsung 65-inch 4K Smart TV
- €129 - Nike Air Max 90
- €89 - Levi's 501 Original Jeans
- €349 - KitchenAid Stand Mixer
- €699 - Dyson V15 Vacuum
- €29 - The Irish Cookbook
- €24 - Guinness Pint Glass Set
- €159 - Fitbit Charge 6
- €329 - Bose QuietComfort Headphones

Notifications:

- 1 - iPhone 15 Pro Max
- 26 - CEO Retail Leadership: Strategic Management
- 30 - iPhone 15 Pro Max

Figure 6: store's index page on live server

5. Project Testing

None.

6. Media Sources

None.

7. Notes for use of web server

Account login not functioning.

Current account is hard-coded to be first name “user” last name “1”.

user 1 can perform all tasks.